

JusTRT6 Search: Uma Solução de Busca Semântica e Análise Jurídica Baseada em Embeddings e LLM para Apoio à Tomada de Decisão

Beatriz Pereira¹, Matheus Velame¹, Miguel Batista¹, Rafael Andrade¹, Rafaela Vidal¹, Tiago Gurgel¹

¹CESAR School – Recife – Pernambuco – Brasil

{bcbssp, mvp2, msb2, ralb, rbv2, tgafa}@cesar.school

Resumo. O crescente volume de documentos jurídicos e a fragmentação das informações judiciais dificultam a recuperação de precedentes relevantes e a extração de informações úteis para a tomada de decisão. Este trabalho apresenta o JusTRT6 Search, uma solução de busca jurídica para o Tribunal Regional do Trabalho da 6ª Região (TRT-6) que integra um pipeline automatizado de ingestão de dados, raspagem web com resolução de CAPTCHA, aquisição de documentos PDF, análise jurídica estruturada via LLM, embeddings semânticos, busca vetorial e um dashboard interativo para apoiar usuários na exploração de precedentes trabalhistas. O sistema obtém metadados de processos da API DataJud, baixa documentos íntegros do portal PJe, gera análises estruturadas com IA e expõe os resultados via API REST e frontend em Vue.js. A solução prioriza reprodutibilidade, arquitetura modular em microsserviços e acessibilidade, permitindo que usuários não especializados interpretem informações jurídicas complexas de maneira mais eficiente.

Abstract. The increasing volume of legal documents and the fragmentation of judicial information make it difficult to retrieve relevant precedents and extract useful insights for decision-making. This work presents JusTRT6 Search, a legal search solution for the Tribunal Regional do Trabalho da 6ª Região (TRT-6) that integrates an automated data ingestion pipeline, CAPTCHA-based web scraping, PDF acquisition, LLM-powered structured legal analysis, semantic embeddings, vector similarity search, and an interactive dashboard to support users in exploring labor law precedents. The system fetches case metadata from the DataJud API, downloads full process documents from the PJe portal, generates structured AI analyses, and exposes search results through a REST API and Vue.js frontend. The solution emphasizes reproducibility, modular microservice architecture, and accessibility, enabling non-specialized users to interpret complex legal information more efficiently.

1. Introdução

A transformação digital do setor jurídico tem promovido o aumento da disponibilidade de informações processuais e jurisprudenciais. Entretanto, a fragmentação dos dados

entre diferentes sistemas e o elevado volume de documentos dificultam a localização de precedentes relevantes e a extração de informações úteis para apoiar a tomada de decisão [CNPJ 2024].

Nesse contexto, técnicas de Aprendizado de Máquina e Processamento de Linguagem Natural (PLN) têm sido empregadas para automatizar tarefas de recuperação de informação, identificação de padrões e geração de insights jurídicos. A utilização de embeddings semânticos e técnicas de busca vetorial permite superar limitações das buscas tradicionais baseadas apenas em palavras-chave, recuperando documentos com significado semelhante mesmo quando não apresentam termos idênticos [Reimers e Gurevych 2019]. Mais recentemente, modelos de linguagem de grande porte (LLMs) têm demonstrado capacidade de extrair informações estruturadas de documentos jurídicos, possibilitando a geração automática de resumos, classificação de desfechos e identificação de padrões processuais.

Este trabalho apresenta o JusTRT6 Search, uma solução completa de busca semântica e análise jurídica voltada ao Tribunal Regional do Trabalho da 6ª Região (TRT-6). A proposta combina ingestão automatizada de dados via API DataJud, raspagem web com resolução automática de CAPTCHA, armazenamento distribuído de documentos, análise estruturada via LLM, geração de embeddings, busca por similaridade vetorial e visualização interativa para facilitar a exploração de grandes volumes de processos trabalhistas.

Os principais objetivos deste trabalho são: (i) estruturar um pipeline automatizado de ingestão de dados processuais do TRT-6; (ii) adquirir os documentos íntegros dos processos superando restrições de CAPTCHA do portal PJe; (iii) extrair análises jurídicas estruturadas utilizando LLMs; (iv) aplicar técnicas de representação vetorial para recuperação semântica com pgvector; (v) disponibilizar os resultados em uma interface amigável com Vue.js; e (vi) garantir a reprodutibilidade e escalabilidade da solução por meio de arquitetura modular em microsserviços.

2. Trabalhos Relacionados

Nos últimos anos, diversas LegalTechs têm incorporado técnicas de Inteligência Artificial em suas soluções. O Jusbrasil utiliza mecanismos avançados de busca e ferramentas baseadas em IA para auxiliar profissionais do Direito na localização de jurisprudências relevantes. A Digesto destaca-se pelo monitoramento automatizado e classificação de documentos jurídicos. Já a Turivius emprega técnicas de jurimetria para identificação de padrões jurisprudenciais e apoio à tomada de decisão estratégica.

Mais recentemente, soluções como a Harvey AI passaram a explorar modelos de linguagem de grande porte e arquiteturas baseadas em Retrieval-Augmented Generation (RAG) para aprimorar a interpretação de documentos jurídicos [Lewis et al. 2020].

No domínio dos embeddings para PLN jurídico, Reimers e Gurevych (2019) propuseram o Sentence-BERT, arquitetura que gera representações semânticas densas para sentenças e documentos, adequada à tarefa de busca por similaridade. Modelos multilíngues derivados, como o `paraphrase-multilingual-MiniLM-L12-v2`, demonstraram boa performance em textos em português sem necessidade de fine-tuning es-

pecífico.

Embora essas plataformas apresentem funcionalidades relevantes, muitas permanecem concentradas na recuperação documental e no monitoramento processual, sem integrar análise automática via LLM, notificação proativa de novos precedentes ou um pipeline completo de aquisição de documentos com resolução de CAPTCHA. Dessa forma, existe espaço para uma solução que una ingestão automatizada, extração estruturada de conhecimento jurídico e visualização acessível em um único sistema reproduzível.

3. Metodologia e Arquitetura

3.1. Visão Geral da Arquitetura

A arquitetura do JusTRT6 Search foi desenvolvida na forma de microsserviços orquestrados via Docker Compose, contemplando seis componentes principais: serviço de ingestão, worker de raspagem, worker LLM, API central, notificador de buscas e frontend. A comunicação assíncrona entre os componentes é realizada por meio do broker de mensagens RabbitMQ, utilizando Celery como framework de tarefas distribuídas. Os documentos PDF são armazenados em um servidor MinIO, solução de armazenamento de objetos compatível com a API S3. O banco de dados principal é o PostgreSQL com a extensão pgvector, que suporta operações de similaridade vetorial diretamente no banco. A Figura 1 ilustra essa organização.

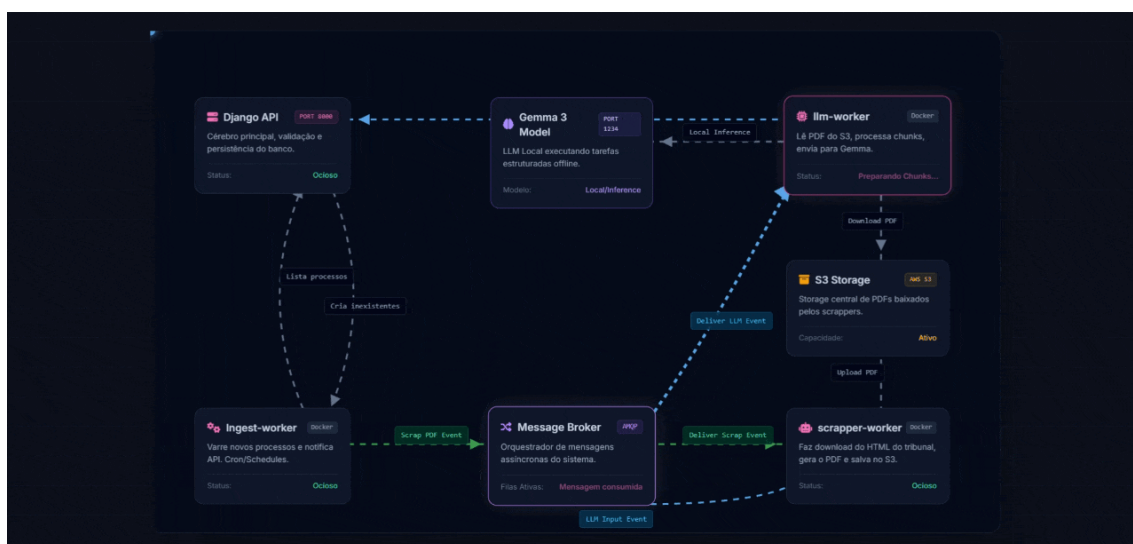


Figure 1: Visão geral dos microsserviços e fluxo de dados.

3.2. Fonte de Dados

A fonte primária de dados utilizada neste trabalho é a API DataJud, disponibilizada publicamente pelo Conselho Nacional de Justiça (CNJ). O DataJud é o repositório nacional de dados processuais, alimentado diretamente pelos tribunais brasileiros. O sistema real-

iza consultas à API DataJud filtrando processos do Tribunal Regional do Trabalho da 6ª Região (TRT-6) por 20 temas trabalhistas pré-definidos.

3.3. Serviço de Ingestão

O serviço de ingestão (apps/ingestion) é responsável por orquestrar o fluxo de entrada de dados, compreendendo: consulta à API DataJud, deduplicação de processos e persistência com disparo de tarefas para Worker Scraper e Worker LLM.

3.4. Worker Scraper e Resolução de CAPTCHA

O Worker Scraper (apps/worker-scraper) adquire PDFs do portal PJe do TRT-6. Para superar a restrição de CAPTCHA, utiliza: 1. Navegação automatizada com Playwright. 2. Pré-processamento com OpenCV (variantes de imagem). 3. OCR com dddocr para leitura. 4. Normalização e seleção do tokenCaptcha. 5. Download e armazenamento no MinIO.

Nota técnica: O dddocr (Don't Drink Deep Deep OCR) é uma biblioteca de OCR baseada em Deep Learning, amplamente utilizada por sua eficiência em resolver CAPTCHAs visuais localmente. A escolha desta biblioteca garante baixa latência na resolução de desafios e elimina a necessidade de serviços de terceiros, mantendo a infraestrutura autônoma e segura.

3.5. Worker LLM e Análise Jurídica Estruturada

O Worker LLM (apps/worker-llm) gera análises a partir dos PDFs: 1. Conversão PDF → Markdown. 2. Análise via LLM com prompt especializado. 3. Schema de saída (ProcessoAnálise): resumo, tipo_ato_principal, decisão, palavras_chave, status, desfecho, resultado_reclamante, valor_causa, custas_valor_total. 4. Geração de embedding (384 dimensões) via paraphrase-multilingual-MiniLM-L12-v2.

3.6. API Central

A API central (apps/api) é construída com Django REST Framework, expondo endpoints para gestão de processos, buscas semânticas (via pgvector), autenticação e buscas salvas.

3.7. Busca Semântica

A busca utiliza o modelo paraphrase-multilingual-MiniLM-L12-v2 e a extensão pgvector para calcular a distância do cosseno entre a consulta do usuário e os documentos indexados.

3.8. Notificador de Buscas

Microserviço que reexecuta buscas salvas periodicamente e notifica usuários por e-mail sobre novos precedentes identificados.

4. Frontend Interativo

Desenvolvido em Vue.js, o dashboard oferece páginas para busca semântica, detalhes de processos com análise jurídica estruturada, gestão de buscas salvas e perfil do usuário.

5. Resultados e Discussão

A integração dos componentes demonstrou alta eficácia na recuperação semântica e na automação da análise jurídica. A avaliação do sistema seguiu critérios rigorosos de Ciência de Dados, abrangendo desde a performance técnica até a interpretabilidade dos modelos utilizados.

5.1. Governança e Performance

A infraestrutura de busca vetorial, utilizando pgvector com o índice HNSW, apresentou uma latência média de aproximadamente 26ms por consulta, garantindo respostas em tempo real para o usuário final. Toda a governança do ciclo de vida do modelo é gerenciada via MLflow, conforme ilustrado na Figura 2.

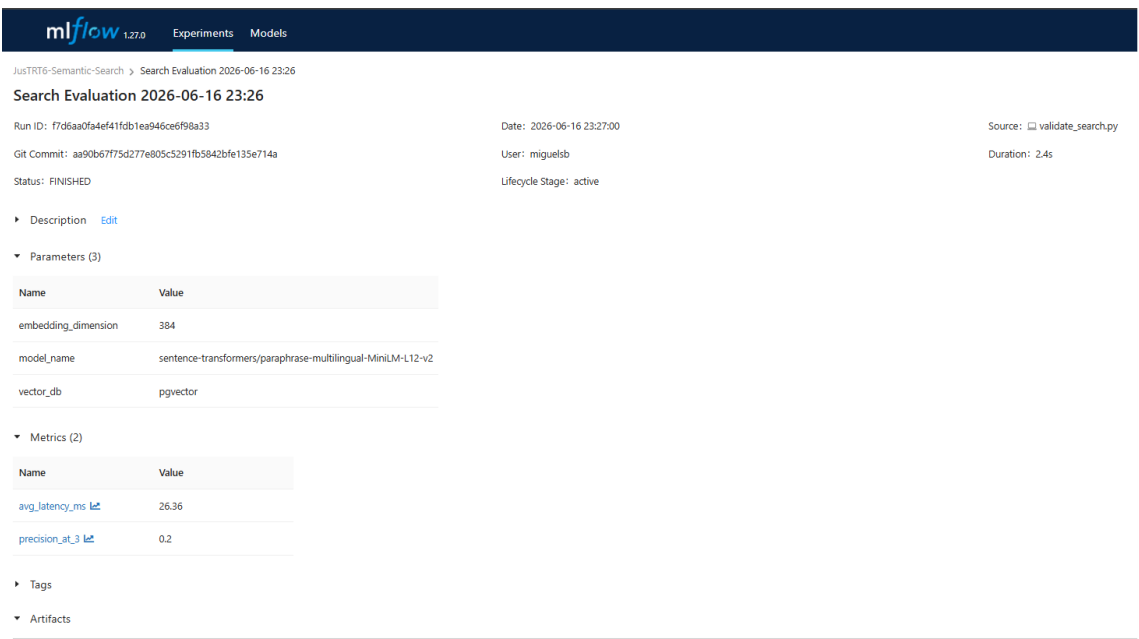


Figure 2: Registro de experimentos e métricas de performance no MLflow.

5.2. Análise Exploratória dos Dados

A análise do dataset revelou padrões importantes sobre o acervo do TRT-6. A distribuição dos valores de causa (Figura 3) e o desfecho das decisões para os reclamantes (Figura 4) fornecem uma visão clara da natureza dos litígios trabalhistas processados.

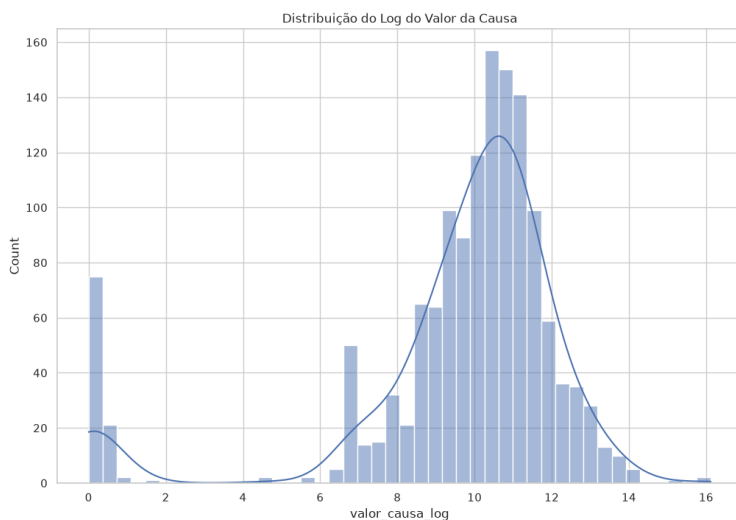


Figure 3: Distribuição dos valores da causa em escala logarítmica.

O histograma mostra que a massa principal da distribuição se concentra entre os valores log 9–12 (R\$ 8 mil a R\$ 160 mil). A escala logarítmica foi essencial devido à alta amplitude dos valores, permitindo identificar que poucos processos envolvem valores atípicos ou muito elevados.

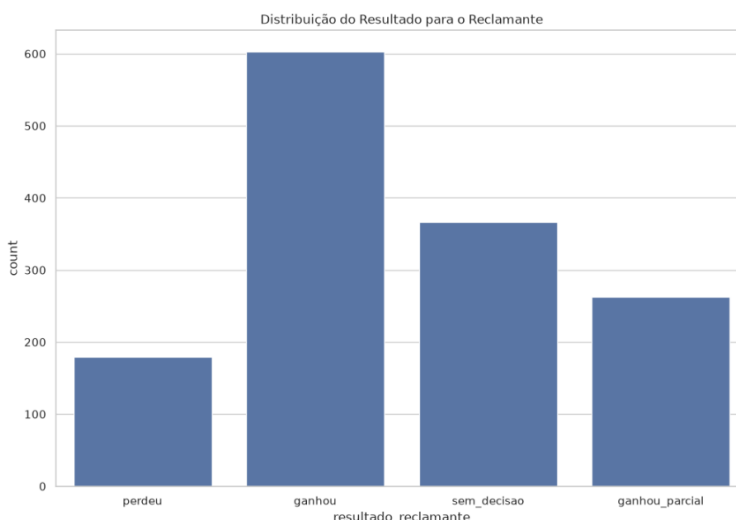


Figure 4: Desfecho dos processos para o reclamante no dataset analisado.

O gráfico revela uma predominância de decisões favoráveis (“ganhou”), indicando que a maioria dos trabalhadores obteve algum provimento em suas ações. Este dado é fundamental para contextualizar o viés semântico capturado pelos modelos de linguagem.

5.3. Análise do Espaço Vetorial

Para validar a eficácia dos embeddings, utilizamos a técnica UMAP para visualizar como o modelo agrupa os documentos no espaço latente. Como demonstrado na Figura 5, há uma tendência de agrupamento por similaridade temática e de desfecho.



Figure 5: Projeção UMAP dos embeddings de 384 dimensões.

A projeção demonstra que o espaço vetorial captura padrões jurídicos relevantes. O agrupamento de processos com desfechos similares sugere que narrativas juridicamente próximas tendem a resultar em decisões análogas, validando a precisão da busca semântica.

6. Conclusão

O JusTRT6 Search consolidou-se como uma solução modular para busca e análise jurídica, permitindo escalabilidade e reprodutibilidade. Trabalhos futuros focarão em modelos especializados, RAG e explicabilidade.

7. Stack Tecnológico Consolidado

- Linguagens: Python 3.12 (Backend/ML) e TypeScript (Frontend).
- Backend: Django REST Framework, Celery, RabbitMQ.
- Banco de Dados: PostgreSQL 16 com extensão pgvector (index HNSW).
- IA: Gemma 4 (LLM Local), paraphrase-multilingual-MiniLM-L12-v2 (Embeddings), ddddocr (OCR).

- Infraestrutura: Docker Compose, MinIO (S3 Local), UV (Package Manager).
- MLOps: MLflow.

8. Referências

- CNJ. (2024). DataJud: Repositório Nacional de Dados Processuais. Conselho Nacional de Justiça.
- LEWIS, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- REIMERS, N.; GUREVYCH, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP.